TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

CONNECTHUB: A MULTI-CLIENT LAN CHAT AND FILE SHARING APPLICATION

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF ENGINEERING IN ELECTRONICS AND COMPUTER ENGINEERING

Submitted by:
Prabesh BC (080BCT054)
Saroj Rawal (080BCT076)
Subesh Yadav (080BCT084)

Submitted to:
Department of Electronics &
Computer Engineering
Pulchowk Campus, IOE
Tribhuvan University, Nepal

August, 2026

Abstract

ConnectHub is a working multi-client LAN chat and file-sharing application written entirely in plain C. It uses nothing beyond the C standard library and standard POSIX calls — no external frameworks, no third-party libraries. Every part of the system, from the network layer to the terminal interface, is built from scratch using system calls taught in the undergraduate curriculum: socket, bind, listen, accept, pthread_create, select, read, and write.

The application follows a classic client–server design. A single multithreaded server manages all connected users, chat rooms, and file transfers. Each client gets its own POSIX thread on the server, and shared data is protected by mutexes. On the client side, a single select() loop watches both the keyboard and the network at the same time, so the terminal stays responsive even during a file upload.

All messages follow a custom text-based protocol — pipe-delimited, newline-terminated lines — that is easy to read and debug with any terminal tool. The system supports room-based chat, private messaging, typing indicators, join/leave notifications, room history, SHA-256 password authentication, and bidirectional file transfer with token-based access control.

Keywords: TCP sockets, POSIX threads, select(), concurrent server, LAN chat, file transfer, custom protocol, C programming, SHA-256

Acknowledgements

We thank the Department of Electronics and Computer Engineering, Pulchowk Campus, for giving us the opportunity to undertake this project as part of our Bachelor's program.

We are grateful to our project supervisor for the guidance and feedback provided throughout the design and implementation phases. The advice received at key decision points significantly shaped the final outcome.

We also acknowledge the faculty members whose lectures on systems programming, computer networking, and operating systems laid the conceptual foundation for this work. The understanding we gained from those courses directly informed every design choice in ConnectHub.

Finally, we thank the authors of the references listed at the end of this report, whose published work served as an essential guide.

Table of Contents

1. Introduction.....1

1.1 Background.....1

1.2 Motivation.....1

1.3 Scope.....2

2. Problem Statement.....3

3. Objectives.....4

3.1 Main Objective.....4

3.2 Specific Objectives.....4

4. Literature Review.....5

4.1 Theoretical Background.....5

4.2 Related Works.....5

4.3 Gap Analysis.....6

5. System Design and Architecture.....7

5.1 System Overview.....7

5.2 Server Architecture.....8

5.3 Client Architecture.....9

5.4 Wire Protocol Design.....9

5.5 Data Structures.....10

5.6 Module Structure.....11

6. Implementation.....12

6.1 Technology Stack.....12

6.2 Module Breakdown.....12

6.3 Authentication.....13

6.4 Room Management.....13

6.5 Messaging.....14

6.6 File Transfer.....14

6.7 Build System.....15

7. Testing and Validation.....16

7.1 Integration Testing.....16

7.2 Concurrency and Stress Testing.....16

7.3 Wire-Level Verification.....17
8. Results and Discussion.....18
9. Conclusion and Future Work.....19
9.1 Conclusion.....19
9.2 Future Work.....19
10. References.....20

List of Figures

Figure 1 LAN Deployment Topology..... 7

Figure 2 Two-Tier Client–Server Architecture..... 7

Figure 3 Server Thread Model and Locking Strategy..... 8

Figure 4 File Transfer Sequence..... 14

Figure 5 Module Dependency Map..... 11

List of Tables

Table 1 Gap Analysis Summary.....6
Table 2 Client → Server Protocol Messages.....9
Table 3 Server → Client Protocol Messages.....10
Table 4 Server Shared Data Structures.....10
Table 5 Codebase Module Breakdown.....11
Table 6 ConnectHub Technology Stack.....12
Table 7 Concurrency and Stress Test Results.....16
Table 8 Project Objectives Outcome.....18

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CLI	Command-Line Interface
DM	Direct Message
FD	File Descriptor
I/O	Input/Output
IRC	Internet Relay Chat
LAN	Local Area Network
POSIX	Portable Operating System Interface
SHA	Secure Hash Algorithm
TCP	Transmission Control Protocol
TUI	Terminal User Interface
TU	Tribhuvan University
IOE	Institute of Engineering

1. Introduction

1.1 Background

Chat applications look simple to users, but building one from scratch requires solving real problems in networking, concurrency, and protocol design. ConnectHub is the result of doing exactly that — a multi-client LAN chat and file-sharing application written entirely in plain C, without any external library doing the heavy lifting.

The server handles multiple simultaneous clients using POSIX threads. The client is a lightweight terminal program that uses `select()` to watch both the keyboard and the network socket at the same time. Every part of the system — the wire format, the thread model, the locking strategy, the file transfer flow — is visible in the source code and directly corresponds to concepts covered in the undergraduate systems-programming and networking curriculum.

1.2 Motivation

Modern chat and file-sharing tools like WhatsApp or Discord require internet access and rely on remote proprietary servers. In closed environments such as a computer lab or an offline classroom, these tools simply do not work. Beyond the connectivity issue, commercial applications hide all the networking mechanics behind high-level abstractions, so they offer no educational value for someone learning how protocols, sockets, and threads actually work.

ConnectHub was built to address both of these problems:

1. **It works fully offline, within a LAN.** Every device only needs to reach the same router or switch — over WiFi or Ethernet. No internet connection, no external server, no subscription.
2. **It is transparent by design.** Every design decision is visible in the source code. The wire protocol can be read with `netcat`. The thread model can be studied in `server.c`. There are no black boxes.

1.3 Scope

ConnectHub is scoped to a trusted academic LAN environment. It includes:

- A concurrent TCP server supporting up to 128 simultaneous clients using POSIX threads and mutexes
- A terminal-based, single-threaded client driven by one `select()` loop
- A custom human-readable text protocol for all client–server communication
- Room-based public chat and private direct messaging
- Real-time typing indicators and join/leave presence notifications
- Per-room message history (last 50 lines replayed on join)
- Token-guarded, base64-chunked file transfer routed through the server
- An admin interface for managing users, rooms, and server statistics

The application does not implement transport encryption. This is a deliberate scope decision for a trusted LAN teaching project and is discussed further in Section 8.

2. Problem Statement

ConnectHub addresses five specific gaps:

3. **No offline LAN communication tool.** Most chat and file-sharing tools require internet access. No lightweight, self-hosted option exists for closed LAN environments like computer labs or offline classrooms.
4. **Black-box protocols.** Commercial messaging applications do not expose their wire format. Students cannot inspect or learn from them.
5. **Teaching examples stop at one client.** Standard beginner socket tutorials handle only a single client at a time. They deliberately skip multi-client concurrency — the most important part from a systems-programming perspective.
6. **Chat and file transfer are taught separately.** Existing course exercises cover either chat or file transfer in isolation. No standard teaching example integrates both into one coherent application.
7. **Terminal clients block on one input.** Simple client examples block on either keyboard input or network data, but not both. Non-blocking `select()`/`poll()`-based clients are rarely demonstrated in practice.

ConnectHub closes all five gaps in a single, working, C-only application.

3. Objectives

3.1 Main Objective

To design and implement a complete, multi-client LAN chat and file-sharing application in plain C using only standard POSIX APIs, demonstrating applied understanding of socket programming, concurrent server design, protocol engineering, and systems-level I/O multiplexing.

3.2 Specific Objectives

8. Design a lightweight, text-based wire protocol for all client–server communication.
9. Implement a concurrent TCP server using POSIX threads with mutex-protected shared state.
10. Build a single-threaded terminal client using `select()` for simultaneous keyboard and network I/O.
11. Support room-based public messaging and private direct messaging.
12. Implement password authentication using a SHA-256 implementation written from scratch.
13. Enable bidirectional file transfer between clients, routed through the server in base64-encoded chunks with per-transfer token validation.
14. Provide typing indicators and user presence notifications.
15. Build an admin interface for user and room management.
16. Validate correctness through integration tests, concurrency stress tests, and wire-level inspection.

4. Literature Review

4.1 Theoretical Background

ConnectHub draws on three main areas of systems-programming theory.

TCP/IP socket programming. The BSD socket API — `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()` — is the backbone of virtually all networked software [4]. TCP guarantees ordered, reliable delivery, which is essential for a chat and file transfer application where messages must arrive in sequence [5]. In the UNIX model, a socket is

simply a file descriptor, which means the same `read()`/`write()` calls used for files work for network connections too [3].

Concurrent server design. A server that needs to handle many clients at once typically spawns one POSIX thread per accepted connection [3, 6]. Threads within the same process share memory — including the user list, room tables, and transfer state — and all shared data must be protected by mutexes to prevent race conditions and deadlocks.

I/O multiplexing. The `select()` system call lets a single-threaded program monitor multiple file descriptors at once and react to whichever is ready first [4]. ConnectHub uses this on the client to watch both `stdin` and the server socket simultaneously, eliminating the need for a second thread. For the protocol itself, line-delimited text formats (as used by IRC [7]) are simple to implement, human-readable, and easy to debug — a natural fit for an educational project.

4.2 Related Works

A review of existing chat implementations in the literature and standard course material shows a consistent pattern of partial solutions:

- **Basic socket tutorials** [8] teach TCP connection setup and echo servers, but stop at one client. No concurrency, no rooms, no file transfer.
- **Multi-threaded chat demos** add per-client threads and broadcasting, but typically support only a single channel with no private messaging or room management [6].
- **IRC** [7] is a well-documented, production-grade line-based protocol that has channels and private messages. It served as a primary design reference for ConnectHub's own protocol, though ConnectHub's version is considerably simpler.
- **Standard course exercises** — echo servers, concurrent TCP servers, `select()`-based clients — provide the individual building blocks that ConnectHub combines into one complete application [4].

4.3 Gap Analysis

Gap	ConnectHub's Answer
Single-client only examples	Concurrent server with up to 128 clients

Gap	ConnectHub's Answer
No room/channel support in demos	Named rooms with access control and history
No private messaging in demos	Full DM system between any two users
Chat and file transfer taught separately	Integrated in one application
Blocking terminal clients	`select()`-based non-blocking client
Black-box protocols	Custom human-readable text protocol

5. System Design and Architecture

5.1 System Overview

ConnectHub follows a two-tier client–server design over TCP. The server is the single source of truth: it holds the connected-user list, registered accounts, active rooms, message history, and in-progress file transfers. Multiple client processes connect to it, authenticate, and then exchange messages and files through the server as an intermediary. There is no peer-to-peer component — all traffic flows through the server.

Figure 1 — LAN Deployment Topology. ConnectHub runs entirely on the local network. Devices may connect over WiFi or Ethernet; no internet connection is required at any point.

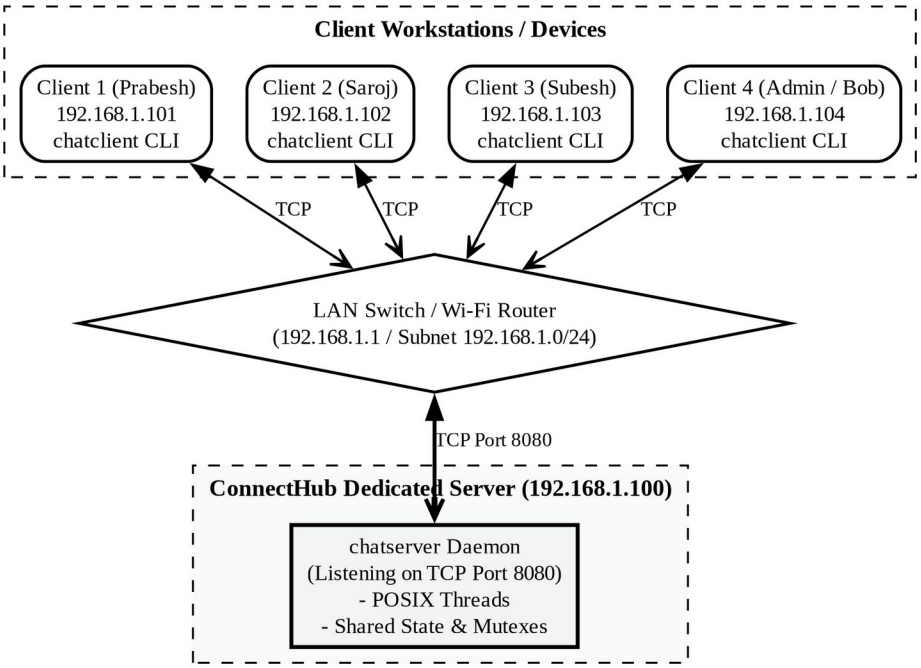

Figure 1: LAN Deployment Topology

Figure 2 — Two-Tier Client–Server Architecture. Every message travels to the server first; the server routes it to the intended recipient(s).

Figure 2: Client–Server Architecture

5.2 Server Architecture

The server (`bin/chatserver`) is a multithreaded TCP server. At startup it loads credentials, initializes subsystems (logger, room manager, user table, history), sets up signal handlers for graceful shutdown, binds a listening socket, and enters an accept loop driven by `select()` with a one-second timeout. On each accepted connection it spawns a new detached POSIX thread for that client. On timeout it does housekeeping: expiring stale upload slots and timed-out queue entries.

Figure 3 — Server Thread Model and Locking Strategy. One detached thread per client; all shared state behind dedicated mutexes.

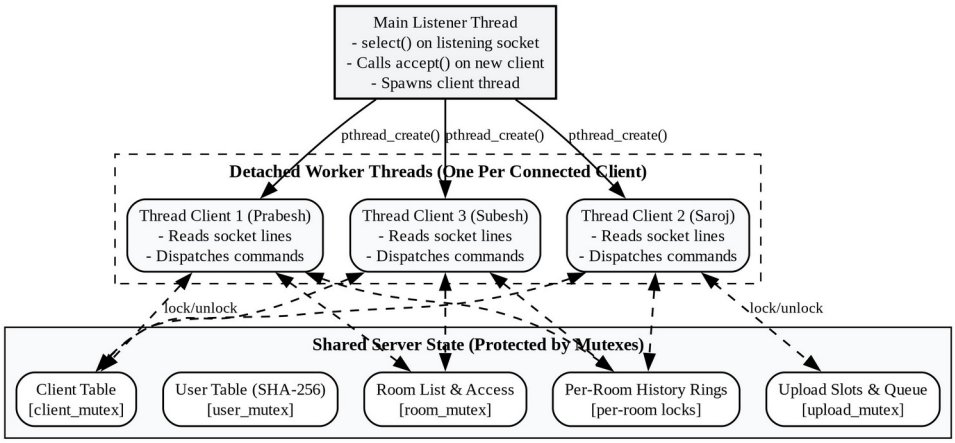

Figure 3: Server Thread Model

The design keeps the server deadlock-free and race-free through four rules:

17. **One mutex per data structure.** The client list, user table, room table, upload slots/queue, and transfer list each have their own lock. Per-room history has its own lock so rooms can be written in parallel.
18. **Never hold `client_mutex` while broadcasting.** Handlers copy what they need under the lock, release it, then send to other clients. This breaks the lock-ordering cycle that could cause deadlock.
19. **Snapshot under lock.** Functions that route data copy the relevant record to a local variable before unlocking, so the caller never touches freed memory.
20. **Cleanup on disconnect.** Every owned resource — upload slots, transfer offers, the client record itself — is released on disconnect, and remaining clients are notified.

5.3 Client Architecture

The client (`bin/chatclient`) is single-threaded by design. Its entire event model is one `select()` loop watching two file descriptors: `stdin` (keyboard) and the TCP socket. `select()` blocks until at least one descriptor is ready, so the client never gets stuck on either source.

The terminal is put into `termios` raw mode with `ICANON` and `ECHO` disabled. The screen is divided into four fixed regions drawn with ANSI escape codes on every loop iteration: a chat

area (scrollback), a sidebar showing online users and rooms, a status bar, and an input line. This gives a responsive full-screen TUI without any curses library.

5.4 Wire Protocol Design

All communication uses a custom, lightweight text protocol. Every message is one ASCII line: fields separated by `|`, terminated by `\n`. The first field is always the message type.

```
LOGIN|prabesh|secret
LOGIN_OK|prabesh
PUBLIC|general|prabesh|Hello everyone!|02:30 PM
PRIVATE|saroj|prabesh|Hi saroj|02:30 PM
FILE_OFFER|saroj|report.pdf|204800|prabesh
```

This line-oriented, human-readable format is the same philosophy as IRC [7]. Because both sides build and parse the same lines, the protocol is fully debuggable with `netcat` or Wireshark without any special tooling. A full list of message types is given in the tables below.

Client → Server messages (key subset)

Message	Fields	Purpose
LOGIN	username, password	Authenticate
PUBLIC	text	Send message to current room
PRIVATE	recipient, text	Send private message
JOIN	room [, password]	Join a room
LEAVE	—	Return to #general
CREATE_ROOM	name, title, desc, password	Create a new room
FILE_REQUEST	filename, size, target	Offer a file to another user
FILE_DATA	filename, token, base64	One file chunk
FILE_END	filename	End of transfer
FILE_ACCEPT	sender, filename	Accept incoming file offer
LOGOUT	—	Clean disconnect

Server → Client messages (key subset)

Message	Fields	Purpose
LOGIN_OK	username	Login accepted
LOGIN_FAIL	reason	Login rejected
PUBLIC	room, sender, text, timestamp	Room message
PRIVATE	sender, recipient, text, timestamp	Private message
NOTIFY	text [, timestamp]	System notification
TYPING	room, username	Typing indicator
FILE_GRANTED	sender, filename, token, size	Upload slot granted
FILE_OFFER	sender, filename, size, target	Incoming file notification
FILE_DATA	sender, filename, base64	Forwarded file chunk
FILE_END	sender, filename	Transfer complete
ERROR	reason	Generic error

5.5 Data Structures

The server keeps the following structures in shared memory, each guarded by its own mutex:

Structure	Contents	Mutex
`clients[]`	sockfd, username, room, thread ID, active flag	`client_mutex`
`users[]`	username → SHA-256 digest	`user_mutex`
`rooms[]`	name, title, description, password hash, members	`room_mutex`
per-room history	linked list of last 50 messages	per-room lock
`upload_slots[2]`	token, sender, file, size, active flag	`upload_mutex`
`upload_queue`	FIFO of pending FILE_REQUESTs (cap: 16 entries, 1 MiB budget)	`upload_mutex`
`transfer_list`	sender, recipient, file, size (for routing FILE_DATA)	`transfer_mutex`

Structure	Contents	Mutex
global counters	total_messages, total_privmsgs, total_files (admin /stats)	—

5.6 Module Structure

The codebase is organized into three directories: `server/`, `client/`, and `shared/`.

Figure 5 — Module Dependency Map.

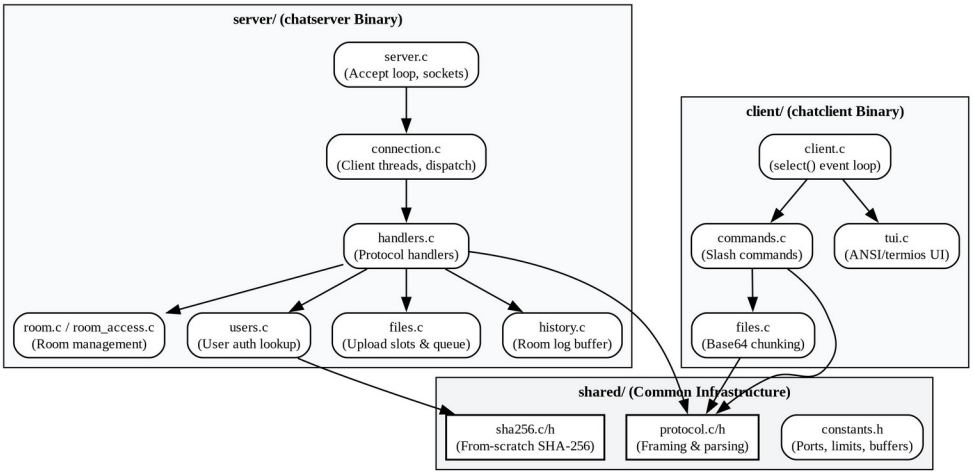

Figure 5: Module Dependency Map

Directory	File	Responsibility
server/	server.c	Entry point, socket setup, accept loop
server/	connection.c	Per-client thread, line reading, command dispatch
server/	handlers.c	One handler function per protocol message type
server/	room.c	Room data structures and management
server/	room_access.c	Room access control (passwords, permissions)
server/	users.c	User account loading and lookup

Directory	File	Responsibility
server/	files.c	Server-side transfer state management
server/	history.c	Per-room ring buffer of recent messages
server/	logger.c	Event logging to logs/server.log
server/	net.c	Low-level socket send helpers
client/	client.c	Entry point, select() loop
client/	commands.c	User command parser (/join, /msg, /sendfile, …)
client/	tui.c	ANSI + termios terminal rendering
client/	net.c	Connect and I/O helpers
client/	protocol.c	Message builder/parser
client/	files.c	Chunking, base64 encode/decode, reassembly
shared/	protocol.c/h	Message-type definitions, framing
shared/	sha256.c/h	SHA-256 implementation
shared/	constants.h	Shared constants (port, buffer sizes, limits)

6. Implementation

6.1 Technology Stack

Concern	Choice
Language	C (C11), compiled with gcc
Networking	Berkeley sockets over TCP (`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`)
Concurrency	POSIX threads; mutexes for shared-state protection (`-lpthread`)
I/O multiplexing	`select()` on both the server accept loop and the client event loop

Concern	Choice
Terminal I/O	`termios.h` raw mode; ANSI escape codes for the TUI
Hashing	SHA-256 implemented from scratch in `shared/sha256.c` — no OpenSSL
Build	Single GNU Makefile; objects in `build/`, binaries in `bin/`
External libraries	None

Key shared constants (`shared/constants.h`): default port `8080`, `MAX_CLIENTS` 128, `BUFFER_SIZE` 4096, `MAX_ROOMS` 32, `MAX_FILE_SIZE` 64 MiB, `FILE_CHUNK_SIZE` 2048 bytes, `ROOM_HISTORY_MAX` 50, `MAX_CONCURRENT_UPLOADS` 2, `MAX_QUEUE_DEPTH` 16.

6.2 Module Breakdown

The codebase has three top-level directories. See Section 5.6 for the full role map.

- **`server/`** — The multithreaded server. `server.c` sets up the listening socket and the accept loop. `connection.c` runs the per-client thread: it reads lines from the socket and dispatches them. `handlers.c` contains one small handler function per protocol message type. `room.c` and `room_access.c` manage room state and access control. `users.c` loads and looks up accounts. `files.c` manages upload slots, the queue, and the transfer list. `history.c` maintains per-room message rings. `logger.c` writes `logs/server.log`. `net.c` provides socket write helpers.
- **`client/`** — The TUI client. `client.c` contains the main `select()` loop. `commands.c` translates user slash-commands into wire messages. `tui.c` renders the four-region terminal UI using raw mode and ANSI codes. `files.c` handles chunking, base64 encoding, and chunk reassembly. `net.c` manages the TCP connection. `protocol.c` builds and parses wire messages.
- **`shared/`** — Code used by both sides. `protocol.c/h` defines message types and framing. `sha256.c/h` is the from-scratch SHA-256 implementation. `constants.h` defines all shared limits and defaults.

6.3 Authentication

When a client connects, it immediately sends `LOGIN|username|password`. The server SHA-256-hashes the provided password and compares it to the stored digest. If they match, the client is marked active, placed in `#general`, and receives a replay of the room's recent history. All other connected clients receive an updated user list. If the credentials are wrong, the server replies with `LOGIN_FAIL` and closes the connection.

A username that is already online is rejected even with correct credentials. This prevents session conflicts without requiring per-session tokens.

Passwords are stored as SHA-256 hex digests in `config/users.cred` and `config/admin.cred`. The SHA-256 implementation is written from scratch in `shared/sha256.c` and follows the NIST FIPS 180-4 specification.

6.4 Room Management

On startup, `#general` is created as the default room. Users join it automatically after login. Users can create additional named rooms with an optional password. The room creator is the owner; only the owner or an admin can delete the room.

A `JOIN` command moves the user from their current room to the target room (after checking the password hash, if protected). Both the old room and the new room receive `NOTIFY` messages announcing the movement. On join, the server replays the last 50 lines of room history to give the new user context. A `/leave` command returns the user to `#general`.

6.5 Messaging

Public messages are sent with the `PUBLIC` command. The server timestamps the message, stores it in the room's history ring, and broadcasts it to all members of the sender's room except the sender themselves.

Private messages are sent with `PRIVATE|recipient|text`. The server delivers the message to the recipient and sends an echo copy back to the sender, so both sides have a consistent conversation record.

Typing indicators are sent as `TYPING|room|username` whenever a user types a character. The client shows "username is typing…" in the sidebar and clears it after a short timeout.

6.6 File Transfer

File transfer uses a server-mediated token scheme that prevents one client from injecting data into another client's transfer. The upload subsystem is rate-limited: a maximum of two concurrent uploads, a per-queue depth of 16, and a combined byte budget of 1 MiB.

Figure 4 — File Transfer Sequence.

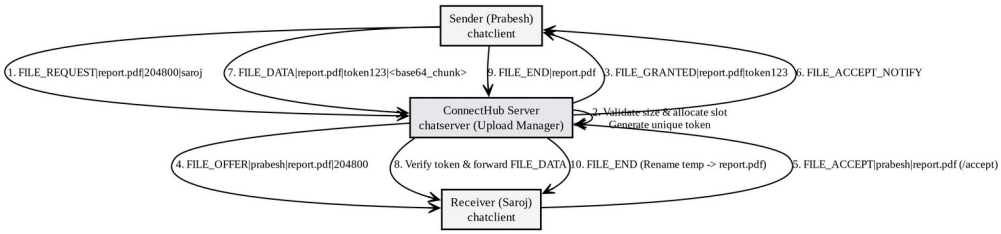

Figure 4: File Transfer Sequence

The steps are:

21. The sender issues `FILE_REQUEST|filename|size|target`. The server validates that the file size is within the 64 MiB limit.
22. The server mints a unique token, reserves an upload slot (or queues the request and replies `FILE_WAIT`), and replies `FILE_GRANTED|sender|filename|TOKEN` to the sender.
23. The server sends `FILE_OFFER|sender|filename|size|target` to the recipient.
24. The recipient runs `/accept` → `FILE_ACCEPT|sender|filename`. The server notifies the sender.
25. The sender streams `FILE_DATA|filename|TOKEN|base64_chunk` messages. Each chunk is about 2 KB encoded in base64. The server verifies the token on every chunk before forwarding.
26. The sender sends `FILE_END|filename`. The server frees the upload slot, forwards the end signal, and the recipient renames the temporary file to its final name.

If the recipient rejects the offer with `/reject`, the slot is released and both sides are notified. Received files are stored in the `files/` directory. Duplicate filenames get a numeric suffix.

6.7 Build System

A single GNU Makefile builds both binaries:

- Object files go into `build/`, mirroring the source directory structure.
- `chatclient` links six client objects plus `shared/protocol.o`.
- `chatserver` links the server objects plus `shared/sha256.o` and `-lpthread`.
- Compile flags: `-Wall -Wextra -O2 -g`.
- `make clean` removes `build/` and `bin/`.

```
make          # build both binaries
./bin/chatserver 8080           # start server
./bin/chatclient 192.168.1.1 8080  # connect a client
```

7. Testing and Validation

7.1 Integration Testing

The project includes `tests/smoke.py`, a Python integration test that connects raw sockets to a freshly started server and drives the full system end-to-end. On a successful run it prints `SMOKE TEST PASSED`. The test covers:

- **Authentication:** valid credentials produce `LOGIN_OK`; wrong credentials produce `LOGIN_FAIL`.
- **Public messaging:** a message sent by one client is received by all others in the same room.
- **Private messaging:** a DM is delivered only to the intended recipient.
- **Room lifecycle:** create, join, and leave a room; verify that presence `NOTIFY` messages go to the right clients.
- **File transfer:** a file offered by one client is received intact by the accepter; the test compares SHA-256 digests to confirm byte-for-byte integrity.
- **Disconnect:** when a client disconnects, the remaining clients receive the correct presence notification.

7.2 Concurrency and Stress Testing

Multiple clients were connected simultaneously and driven to produce high message throughput concurrently. The following checks were performed:

Check	Result
No message duplication, dropping, or field corruption under concurrent load	Passed

Check	Result
No deadlock when multiple clients join/leave the same room simultaneously	Passed
Concurrent file transfers do not interfere with chat traffic	Passed
Clean resource cleanup (file descriptors, thread memory) on disconnect	Passed

No deadlocks, race conditions, or resource leaks were observed in extended sessions. The open file-descriptor count (checked via `/proc/<pid>/fd`) returned to baseline after each test.

7.3 Wire-Level Verification

`tcpdump` and Wireshark captures confirmed:

27. All messages are framed as newline-terminated ASCII strings with no partial or misdelimited frames.
28. File chunks arrive in order with intact base64 content.
29. TCP connections close gracefully (FIN/ACK exchange) on `/logout` or Ctrl-C.
30. The custom protocol is fully human-readable in the capture without any special dissector.

8. Results and Discussion

ConnectHub meets all of its stated objectives. The following summarizes what was achieved and where limitations exist.

What Was Achieved

Objective	Outcome
Concurrent multi-client server	Handles up to 128 simultaneous clients; no deadlock or state corruption observed
Room-based chat	Named rooms with passwords, history replay, and access control work correctly
Private messaging	DMs delivered only to the intended recipient; echo copy to sender
Typing indicators and presence	Notifications sent and cleared correctly

Objective	Outcome
File transfer	Byte-for-byte integrity confirmed by SHA-256 digest comparison in tests
SHA-256 authentication	From-scratch implementation, no OpenSSL dependency
Admin interface	User creation/deletion, password reset, kick, announce, and stats all functional
Zero external dependencies	Builds and runs with only `gcc`, `make`, and a POSIX OS
Debuggable protocol	Protocol fully readable in `netcat` and Wireshark without any special tooling

Limitations

No transport encryption. Credentials and messages travel as plaintext over TCP. This is acceptable for a trusted offline LAN project, but TLS (via OpenSSL or mbedTLS) would be required for any deployment on an untrusted network.

No persistence. Rooms, users created at runtime, and message history all live in memory and are lost on server restart. Only the `config/*.cred` files survive. A persistent storage layer (SQLite or flat-file serialization) would be needed in a production setting.

These two limitations were known from the outset and are intentional scope decisions, not oversights. They are the most natural starting points for future work.

9. Conclusion and Future Work

9.1 Conclusion

ConnectHub shows that a fully functional, feature-rich, multi-client chat and file-sharing application can be built from first principles in plain C using nothing but standard POSIX APIs. The project integrates the core topics of the undergraduate systems-programming curriculum — TCP sockets, POSIX threads and mutexes, `select()`-based I/O multiplexing, custom protocol design, and cryptographic hashing — into one coherent, working system.

The text-based pipe-delimited protocol was simple to implement and easy to debug, confirming why line-oriented protocols have been the standard for text communication

systems since IRC. The per-client-thread server model handled concurrent load without issues at the scale of an academic LAN. The single-threaded `select()`-based client was genuinely simpler and more responsive than a multi-threaded alternative would have been.

Building the system one feature at a time — starting with login and broadcast echo, then adding rooms, DMs, presence, and finally file transfer — kept complexity manageable and ensured each component was tested before the next one was added.

9.2 Future Work

31. **TLS/SSL encryption** — adding OpenSSL or mbedTLS would make ConnectHub usable on untrusted networks by encrypting all traffic.
32. **Persistent storage** — SQLite or flat-file serialization would preserve rooms, accounts, and history across server restarts.
33. **Thread pool with event queue** — replacing the per-client-thread model would improve scalability at high connection counts.
34. **`epoll()` on the server** — replacing `select()` in the accept loop would eliminate O(n) descriptor scanning at scale.
35. **Web or GTK front-end** — a WebSockets bridge or GTK client would make the system accessible to non-terminal users; the shared protocol already supports it.
36. **End-to-end file encryption** — AES encryption on file chunks would prevent the server from reading transferred content.
37. **Delivery acknowledgements** — explicit ACKs would let clients detect dropped or failed deliveries.

10. References

38. I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson, 2016.
39. B. A. Forouzan, *Data Communications and Networking*, 5th ed. New York: McGraw-Hill, 2017.
40. W. R. Stevens, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA: Addison-Wesley, 2013.
41. W. R. Stevens, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA: Addison-Wesley, 2003.

42. J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Boston, MA: Pearson, 2021.
43. D. R. Butenhof, *Programming with POSIX Threads*. Boston, MA: Addison-Wesley, 1997.
44. J. Oikarinen and D. Reed, "Internet Relay Chat Protocol," IETF RFC 1459, May 1993.
45. B. W. Kernighan and R. Pike, *The UNIX Programming Environment*. Englewood Cliffs, NJ: Prentice Hall, 1984.

End of Report — ConnectHub Final Project Report, August 2026